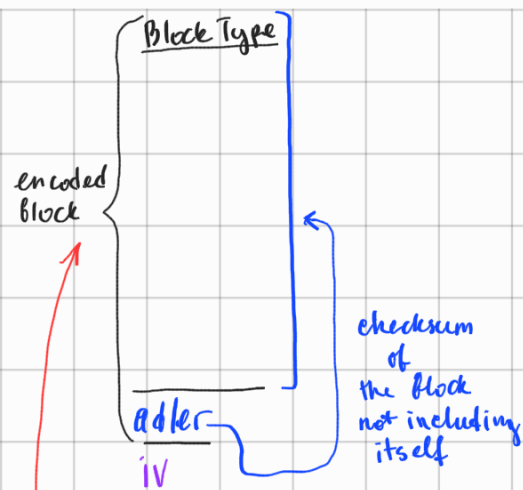




- Базовая структура блока:

• зашифрованный блок; (4064)

• iv и adler checksum 6 байт. $(16+2)=18$

итого 4078

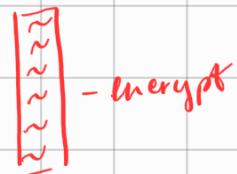
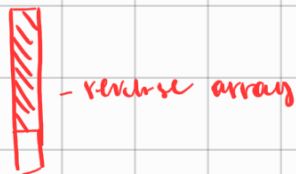
We encode adler16 checksum as well, so that we can quickly tell that we decoded it correctly or not.

Does that reveal anything or does that make it easier to break the encoding?

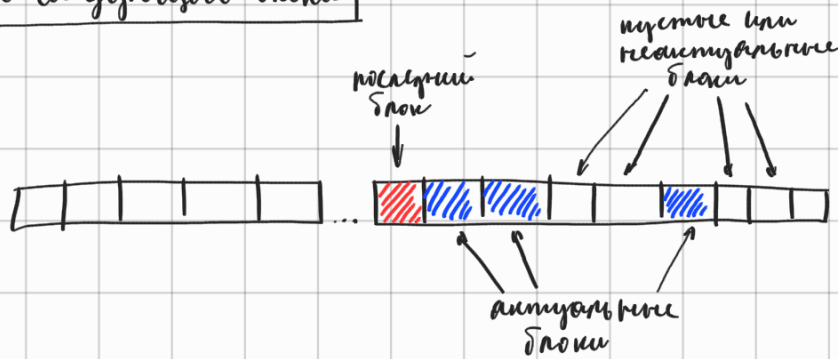
- ⑤ - Encryption key - this block contains the encryption key to decrypt other blocks
- ⑤ - Folders - a tree-like structure of folders and subfolders
- ⑤ - Symbol Sets - sets of symbols for word generation (e.g. lower+upper+digits)
- ⑤ - Phrase Templates - templates for phrases (e.g. 'website' → 'url', 'login', 'password')
- Phrases - instances of templates (e.g. if "website": "gmail" → "mail.ru", "user", "pass")
i.e. actual logins and passwords

- We use AES-256 CBC with IV per block

- Revert Block byte array before encrypting - random bytes go first



Запись следующего блока



Инициализация

• Находим следующие блоки:

1) последний блок

2) следующий активный блок "справа" от блока (1)

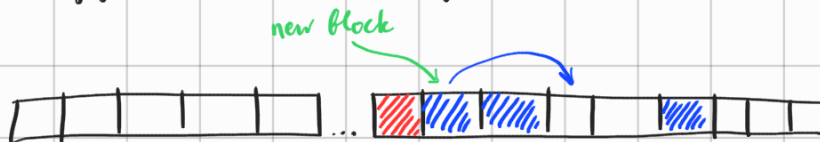
3) следующий неактивный блок "справа" от блока (2)

• копируем блок (2) в блок (3), при этом обновим версию

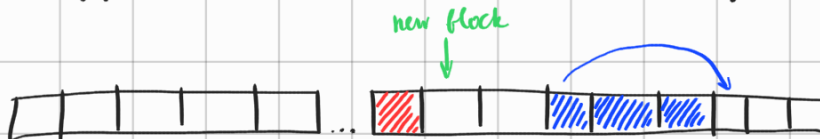
• записываем новый блок в следующий блок после блока (1)

этот блок всегда будет неактивным, т.к. мы переместили блок (2) непосредственно перед этим.

1) Если следующий блок активен



2) Если следующий блок неактивен или пуст



- Это позволяет распределить нагрузку относительно равномерно по всей протяженности флеша, а кроме того предотвращает bit rot, характерный для флеш-памяти, путем копирования активных блоков.

- Можно также реализовать функцию дитту-обновления старейших активных блоков, для вызова пользователем.

Улучшение безопасности

① CleanUp/Depragment

- Удалить все неактивные блоки и оптимизировать структуру.

② Shuffle

- Перемешать блоки для предотвращения атаки на KeyBlock, который без этой функции будет находиться в самом начале.

③ Fake empty blocks

- Заполнить пустые блоки рандомными данными ~~и добавить~~

→ контрольная
сумма
криптуется?

↑
можно добавить
там же шестерки

~~валидную контрольную сумму в footer, где сокращение~~
факта их пустоты и невозможности исключения их из
анализа атакующими.

↑ можно даже создать "валидные" блоки с фальшивыми данными,
зашифровав их функцией хэширования, с правильной IV и итерацией

④ Fake key blocks

- Создать несколько версий KeyBlock, из которых только последняя
будет содержать активные ключи и криптографические активные
параметры. Таким образом, атакующий имеет вероятность
потратить время и ресурсы на неактивный KeyBlock.

table Folder
folder-id: uint16
parent-folder-id: uint16
folder-name: string

Folders

table Folders Block
block: StoreBlock
folders: [Folder]

table Key Block
block: StoreBlock
bucket-count: int
key: [byte]
db-name: [byte]
iv: [byte]

encryption key

table Symbol Set
set-id: uint16
symbol-set-name: string
symbol-set: string

Symbol Sets

table Symbol Sets Block
block: StoreBlock
symbol-sets: [SymbolSet]

table Word Template
word-template-id: uint16
permissions: byte
icon: byte
min-length: uint16
max-length: uint16
word-template-name: string
symbol-set-ids: [uint16]

Phrase Templates

table Phrase Template
phrase-template-id: uint16
phrase-template-name: string
word-template-ids: [uint16]

table Phrase Template Block
block: StoreBlock
word-templates: [WordTemplate]
phrase-templates: [PhraseTemplate]

Phrases

table Word
word-template-id: uint16
name: string
word: string
permissions: byte
icon: byte

table Phrase History
phrase-template-id: uint16
phrase: [Word]

table Phrase Block
block: StoreBlock
phrase-template-id: uint16
folder-id: uint16
is-tombstone: bool
phrase-name: string
history: [PhraseHistory]

struct StoreBlock
block-type: BlockType ^(byte)
block-id: uint16
version: uint32
entropy: uint64